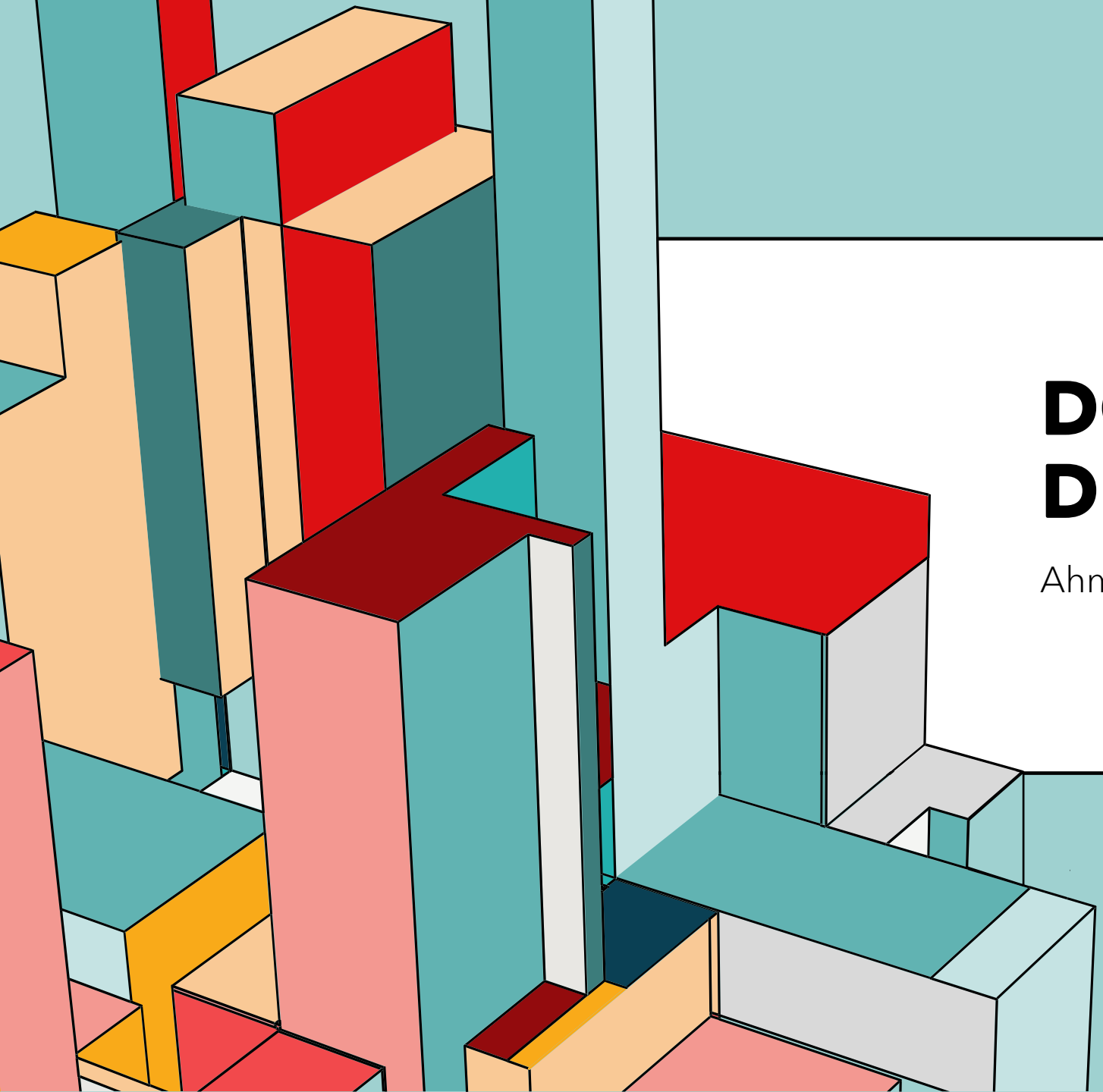
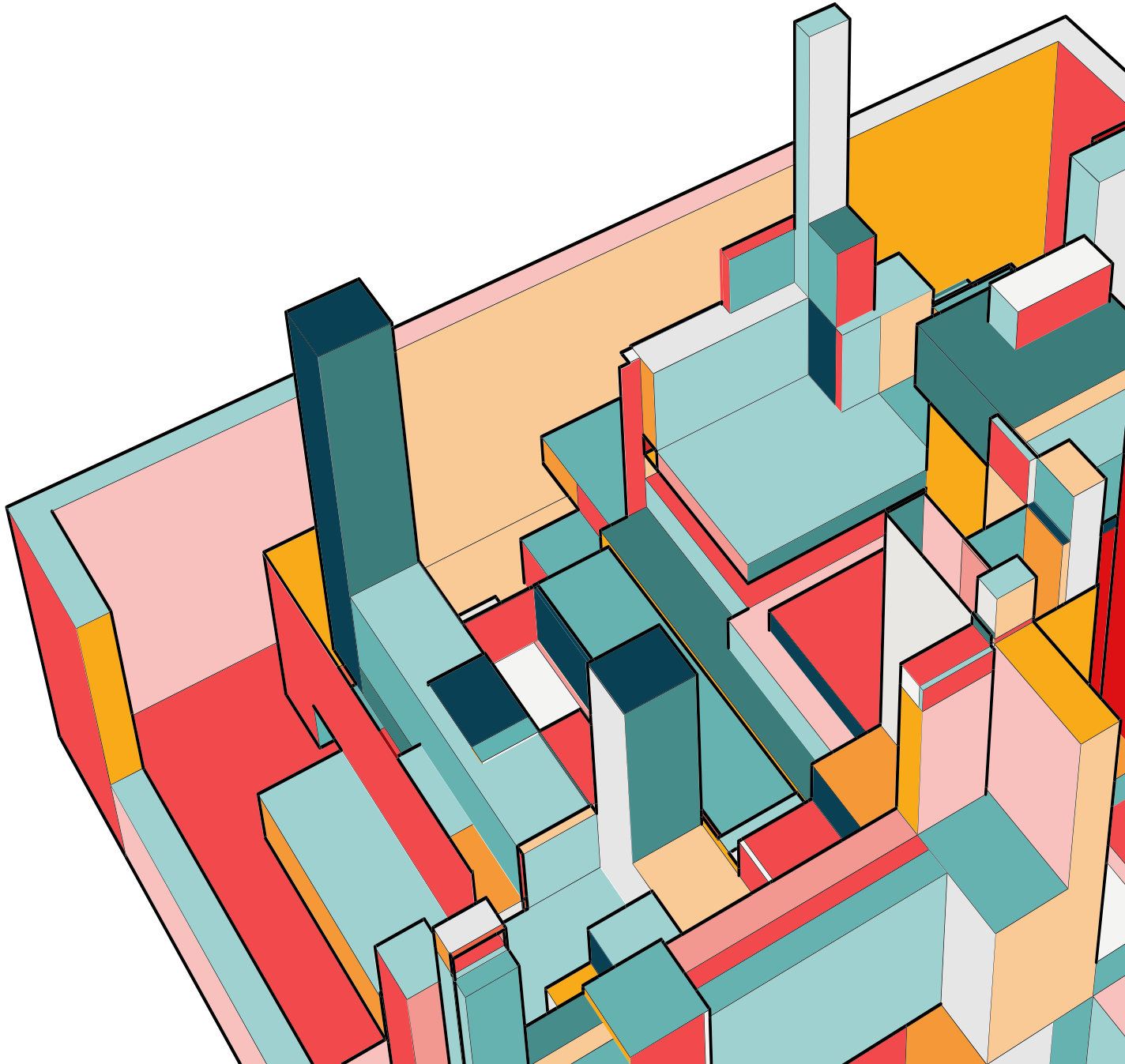


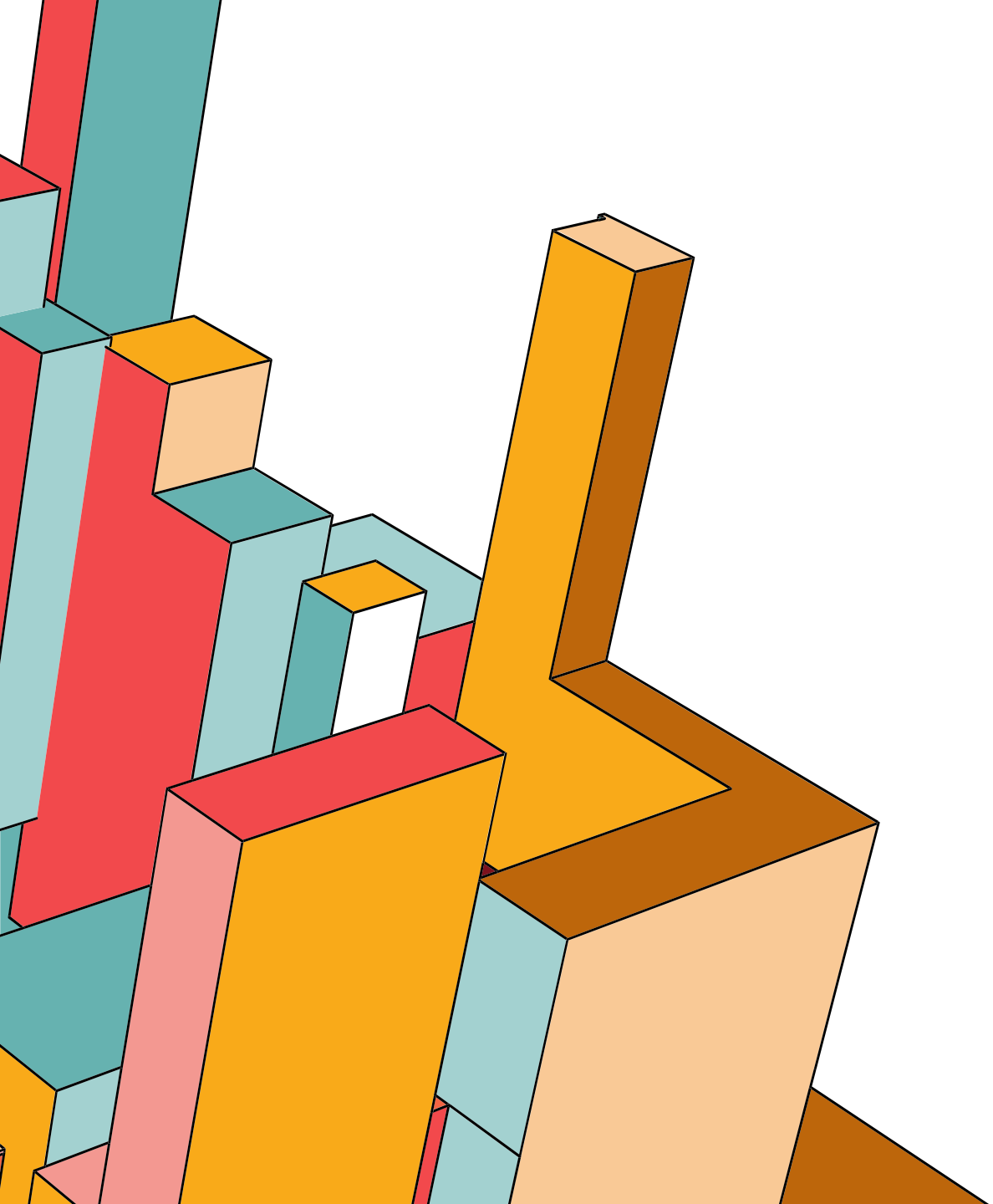
DOMAIN DRIVEN DESIGN

Ahmed Abouzeid

AGENDA

- What is Domain Driven Design?
- Modeling the problem
- Element of DDD

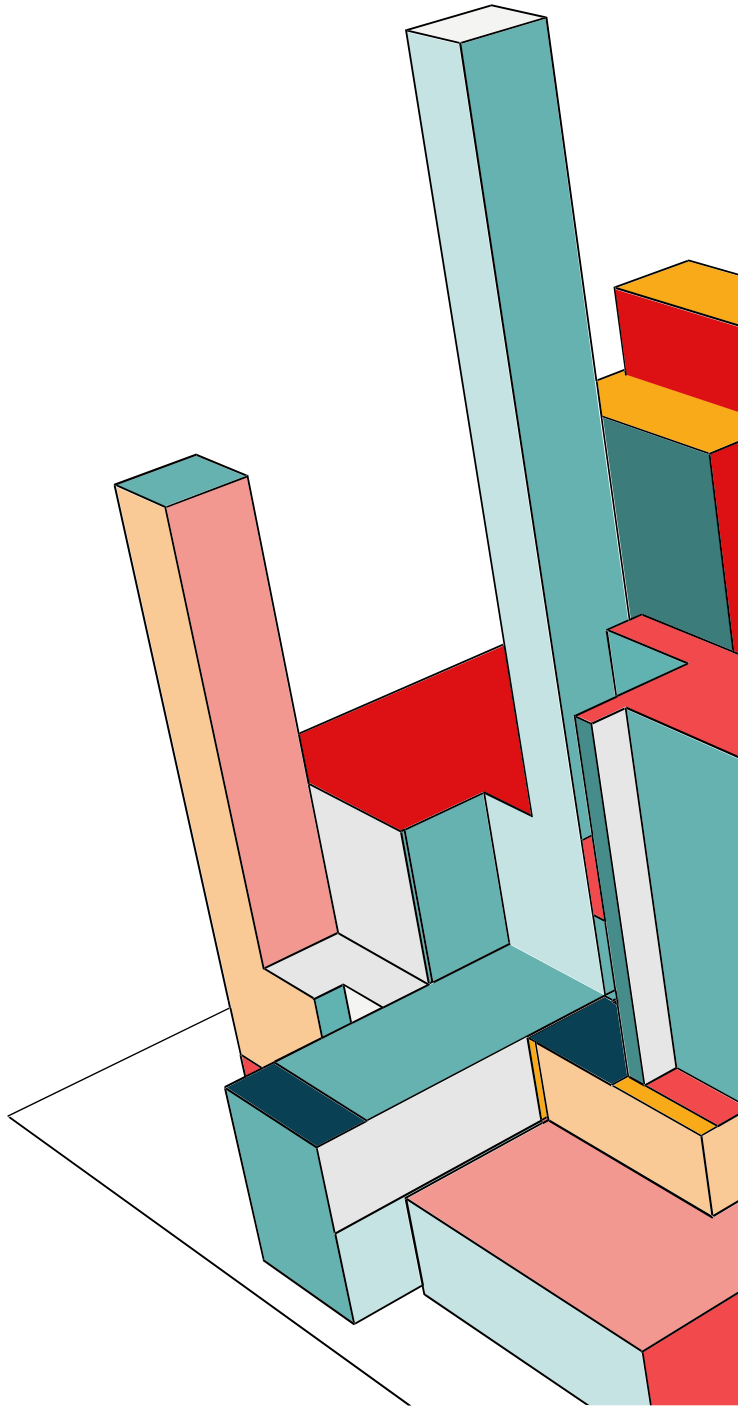




WHAT IS DOMAIN DRIVEN DESIGN?

SOFTWARE COMPLEXITY TYPES

- Technical complexity
- Amount of data
- Performance
- Business logic complexity

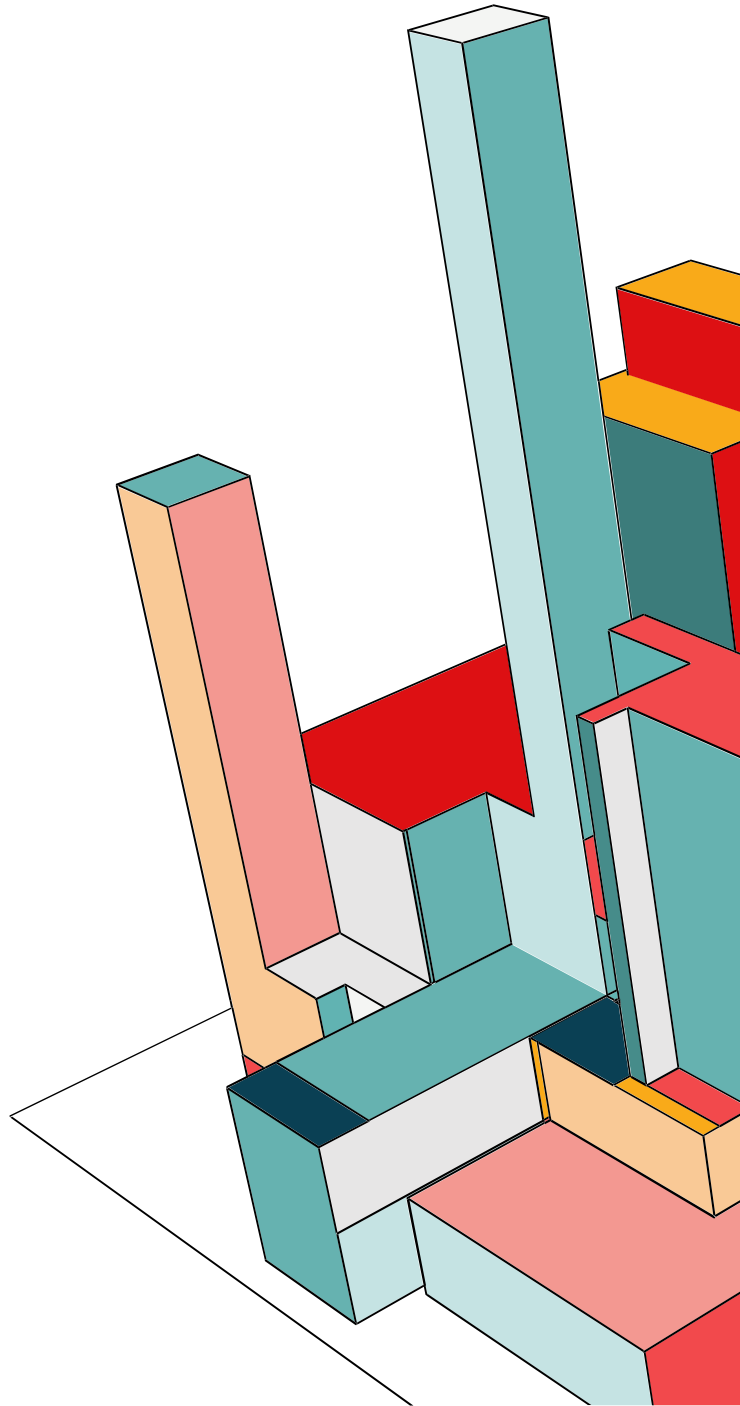


SOFTWARE COMPLEXITY TYPES

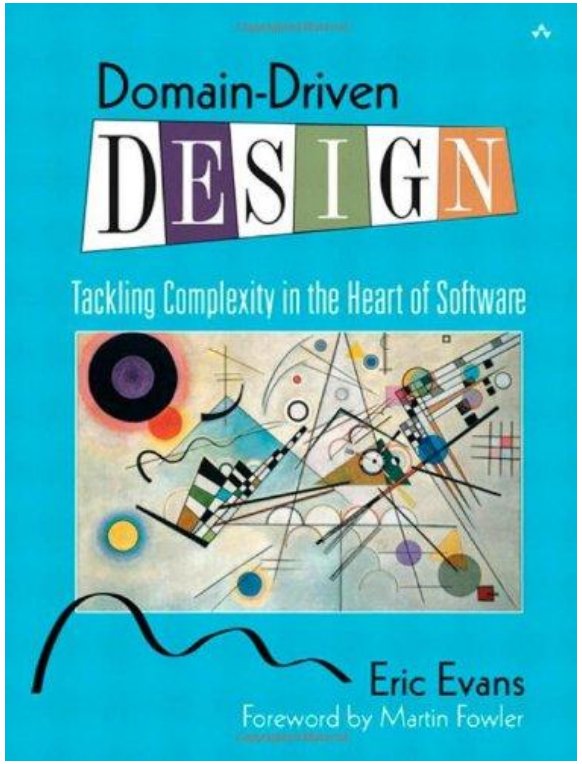
- Technical complexity
- Amount of data
- Performance
- Business logic complexity → DDD

Domain Driven Design can handle complex business logic complexity in such away that it would be possible to extend, maintained and keep it simple

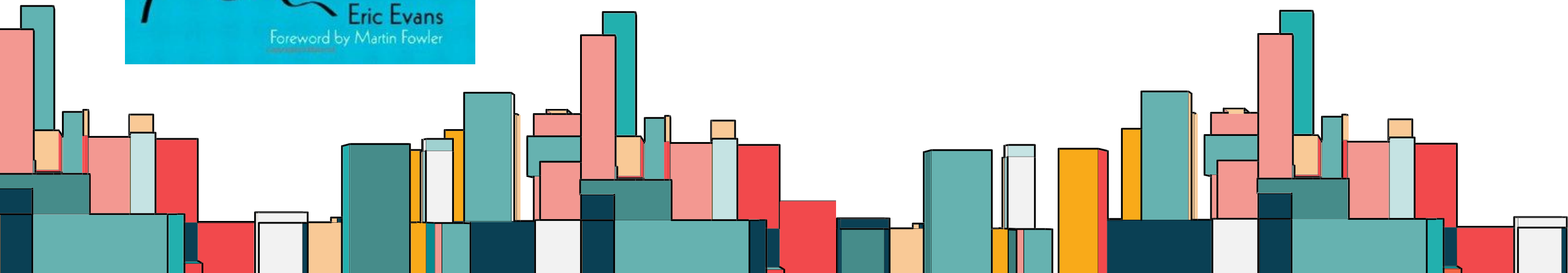
Patterns & Practices



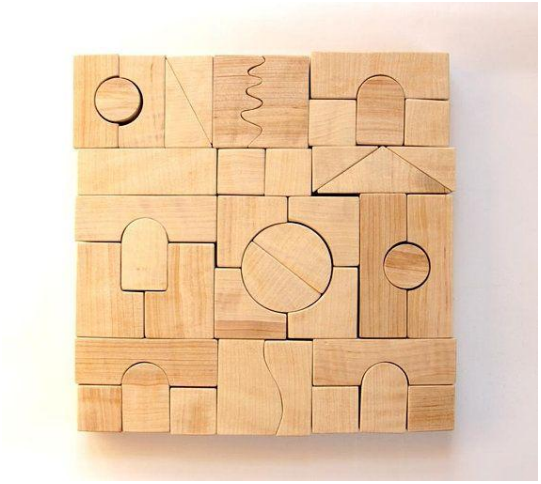
THE BLUE BOOK



Domain Driven Design - Tackling Complexity in the Heart of Software by Eric Evans is the main reference for Domain Driven Design

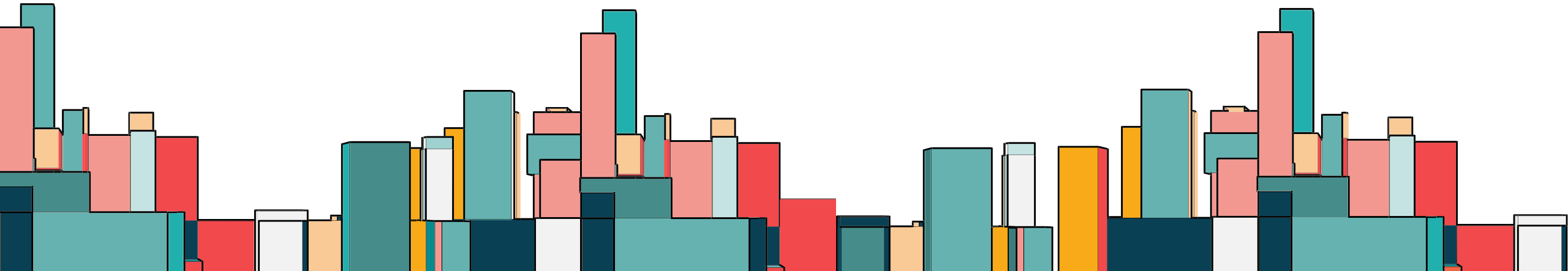


WHAT IS DOMAIN DRIVEN DESIGN ?



Domain
Field of study that defines a set of common requirements, terminology, and functionality for any software program

Domain Driven Design (DDD)
The expansion upon and application of the domain concept
It aims to ease the creation of complex applications by connecting the related pieces of the software into an ever-evolving model



UBIQUITOUS LANGUAGE

Domain Expert

Balance Sheet

Budget

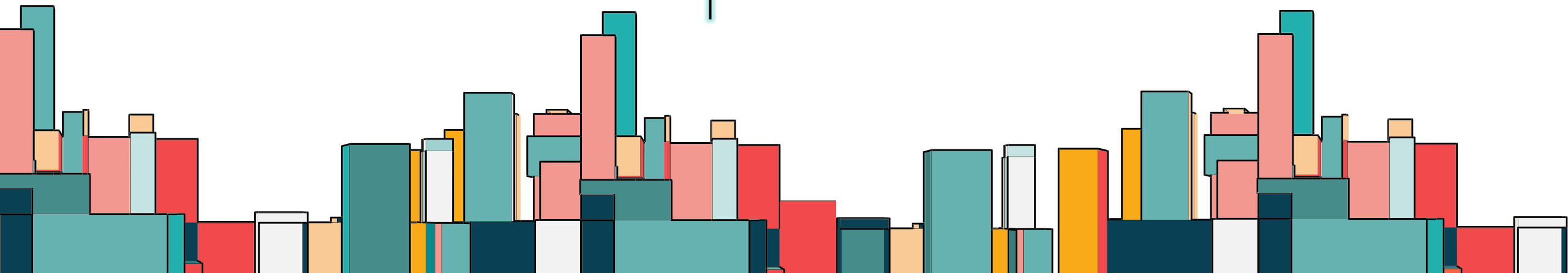
VAT

Classes & Objects

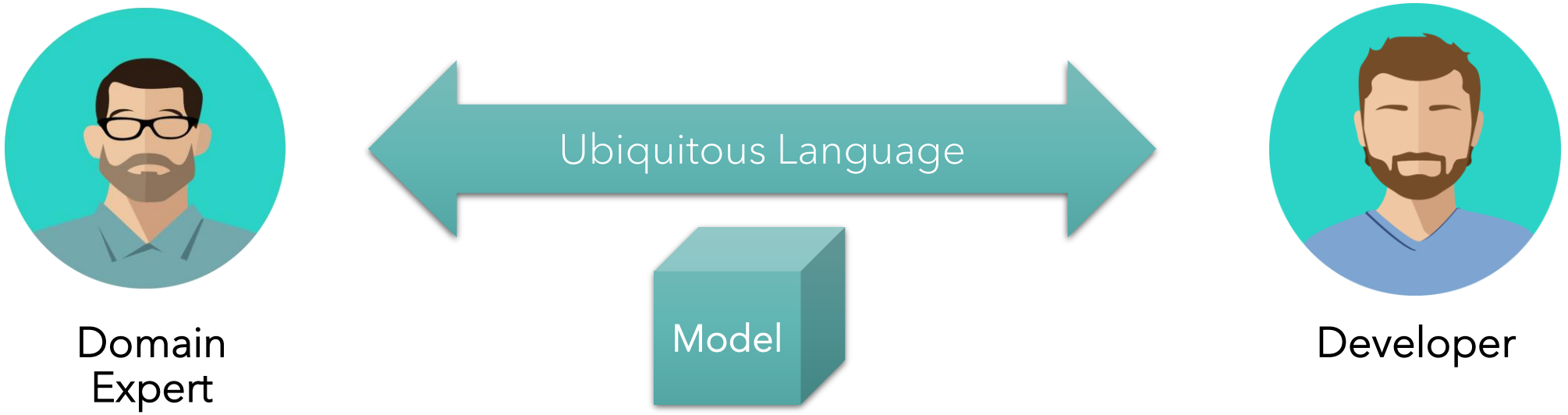
Database Tables

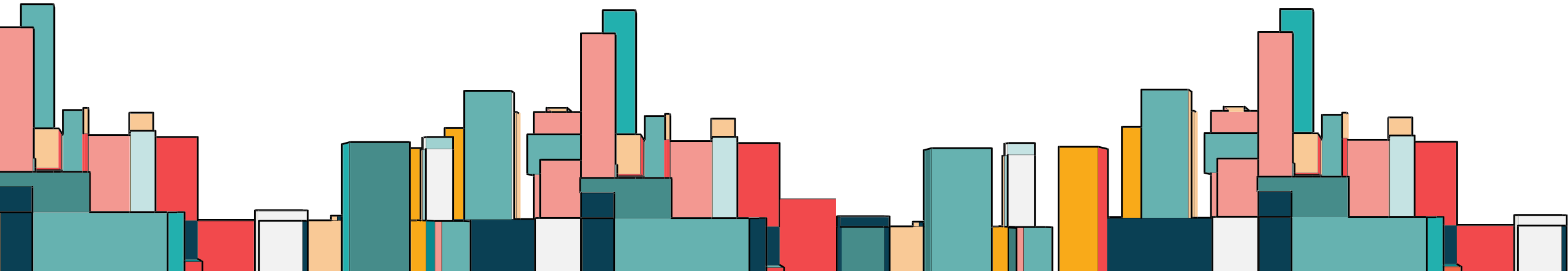
HTML Views

Developer



UBIQUITOUS LANGUAGE

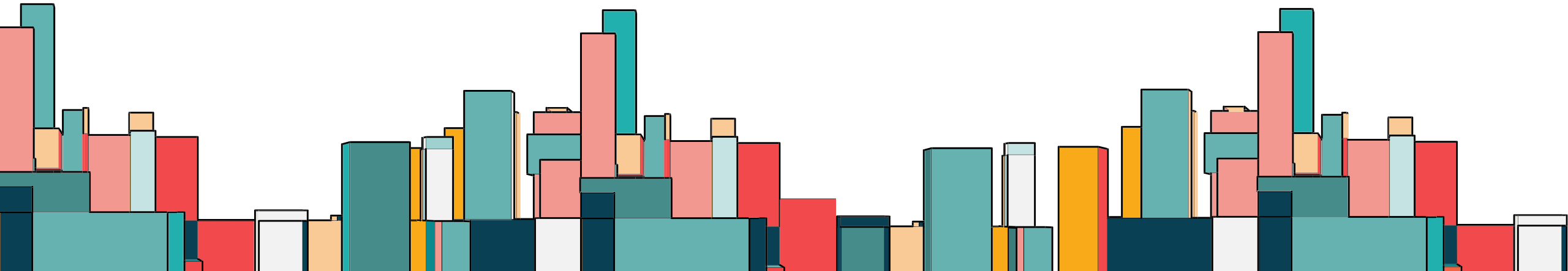


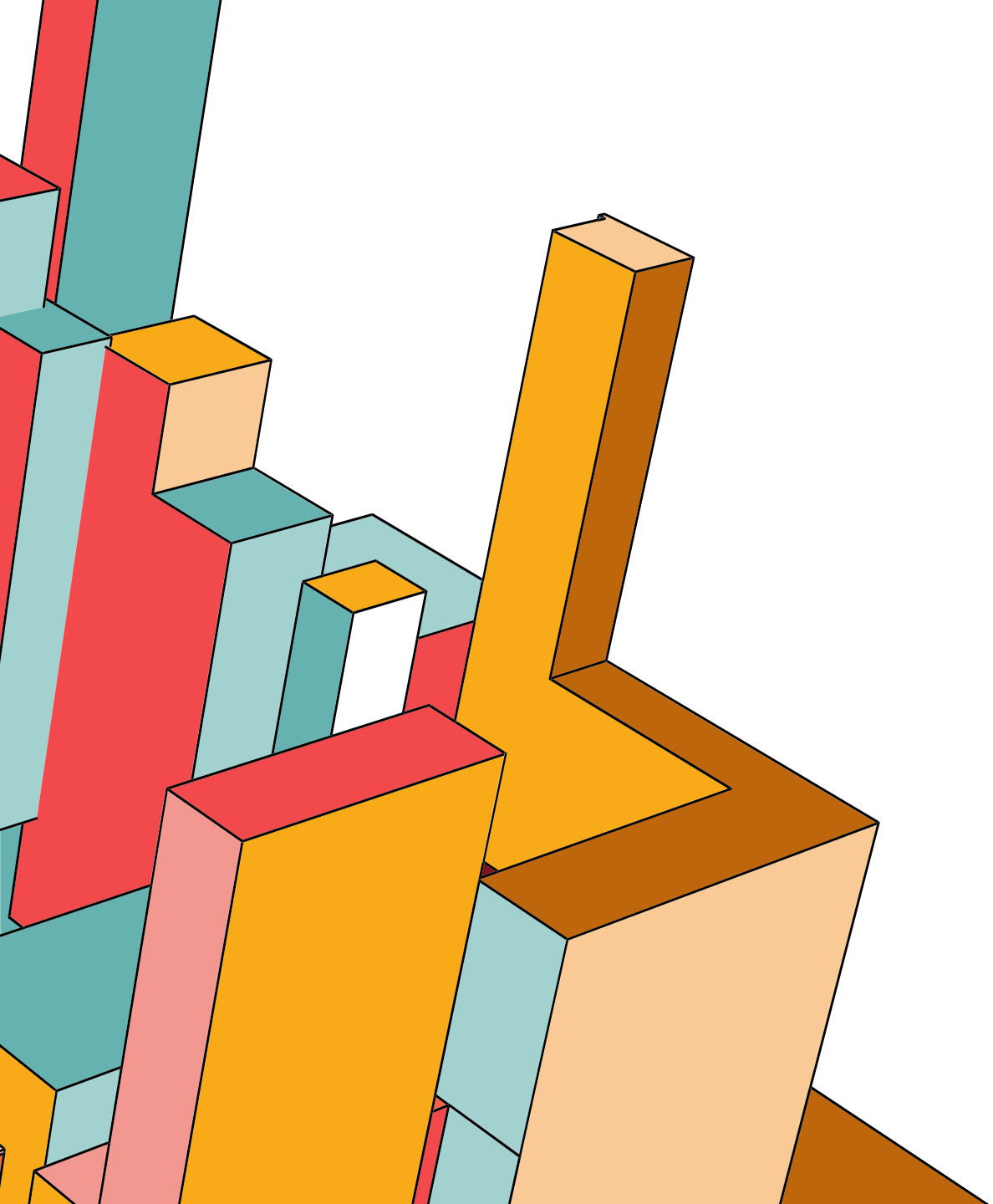


UBIQUITOUS LANGUAGE

CUSTOMER CONVERSATION

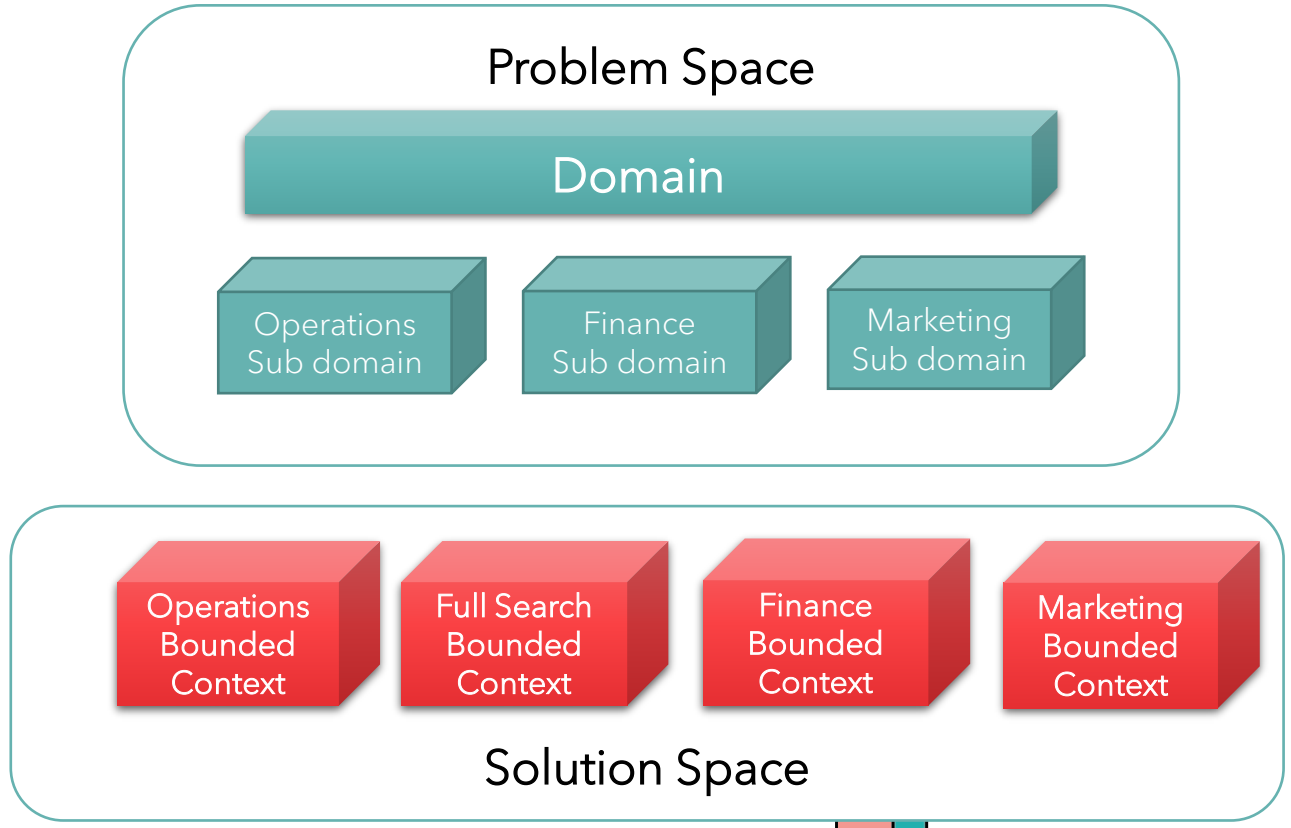
- Important to get on the “same page” with the domain expert
- The customer gained better understanding of their business process by describe it in terms we could understand and model
- Avoid speaking in programmer terms
- Focus on how the domain works, not how the software will work
- Make the implicit knowledge of domain expert explicit





MODELING THE PROBLEM

MODELING THE PROBLEM



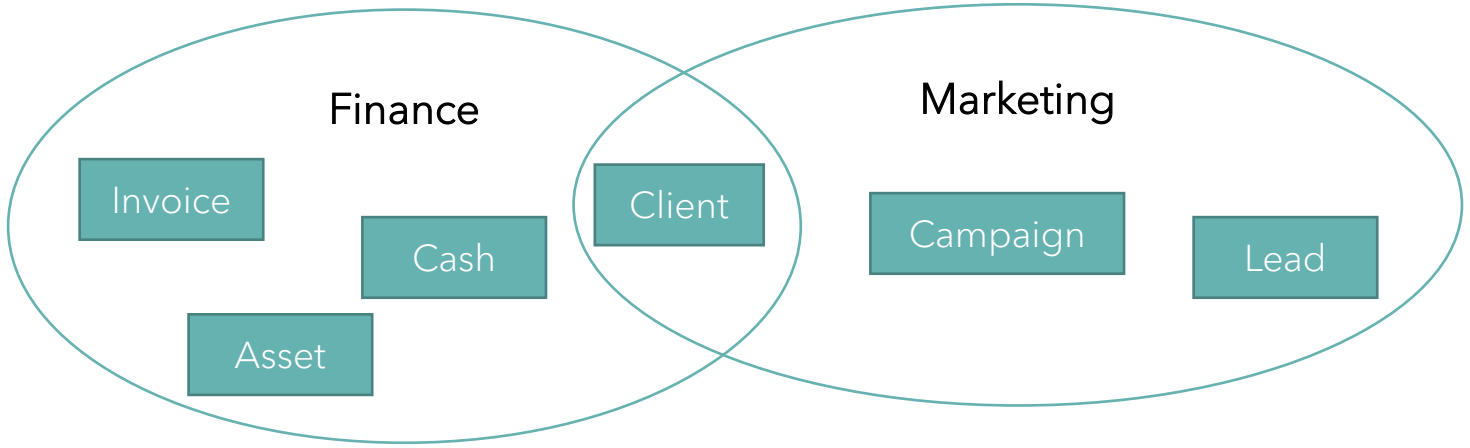
We have to find clear boundaries between different parts of the system

Sub Domain is a problem space concept

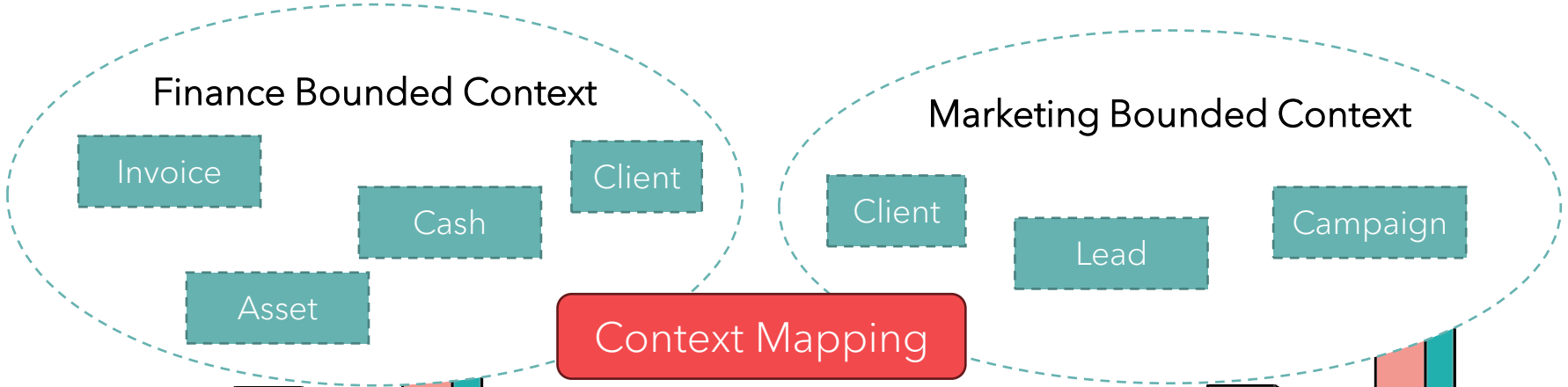
Bounded Context is a solution space concept

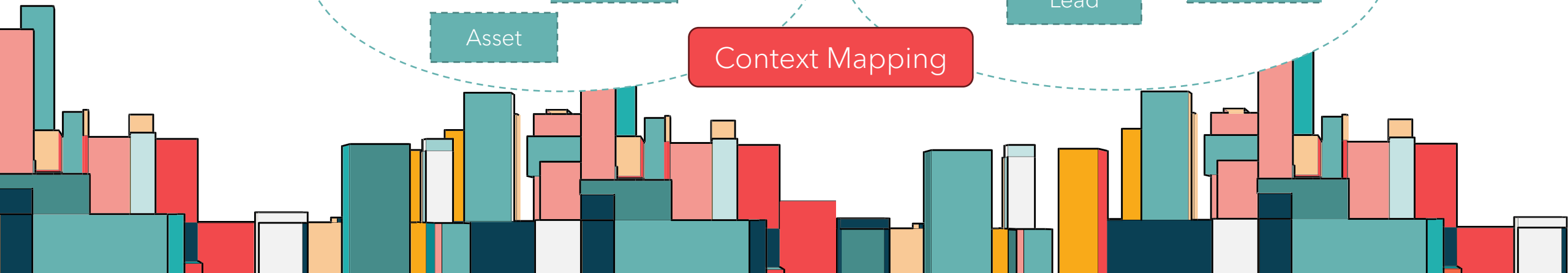
MODELING THE PROBLEM

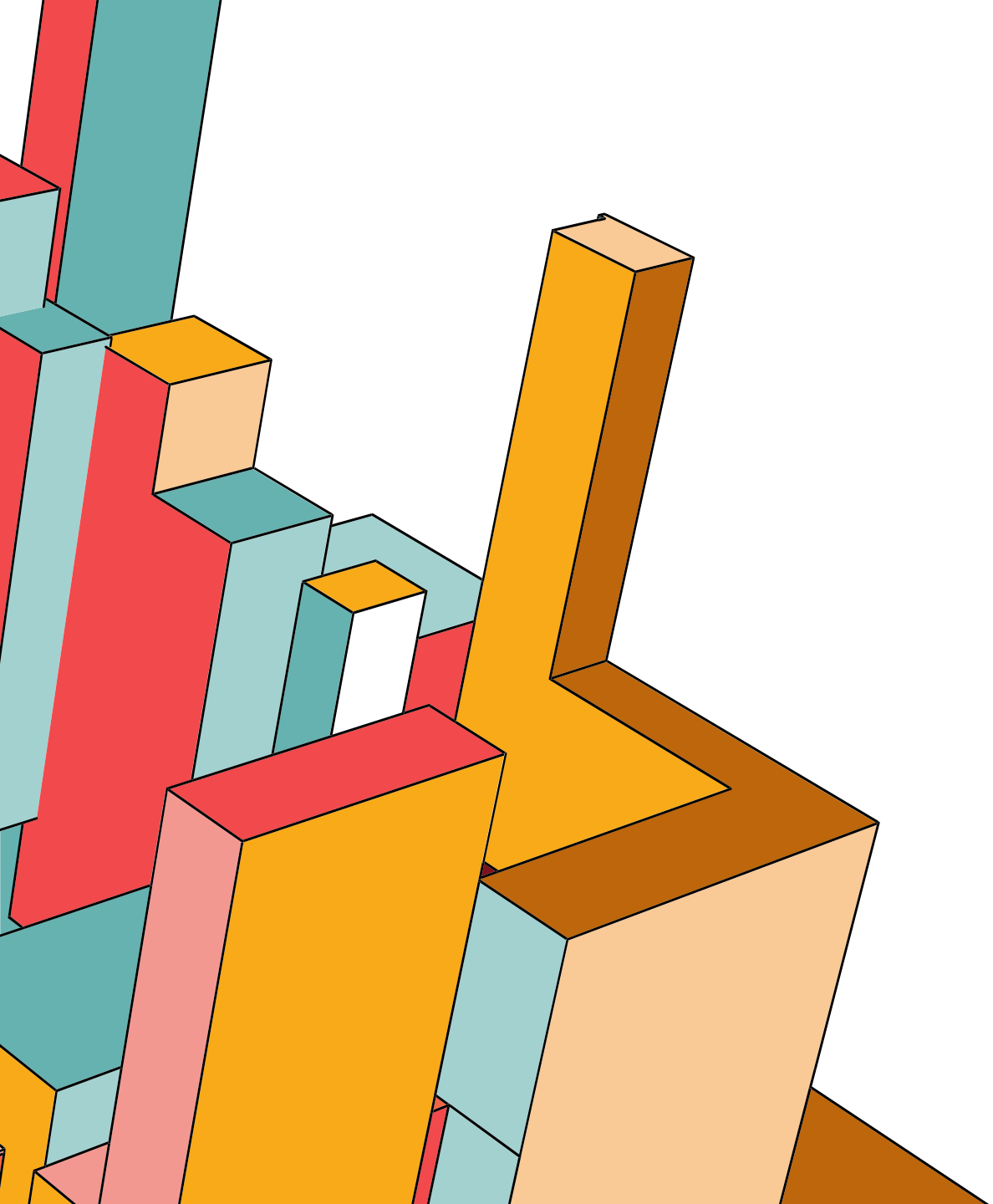
Domain



Bounded Context







ELEMENTS OF DDD

FOCUS ON DOMAIN

- Focus on **Domain Model** not how to persist
- Focus on **behavior** not properties

I want to **purchase** a product

I want to **approve** the workflow request

I want to **schedule** a meetings

I want to **monitor** employees' performance

ANEMIC AND RICH MODELS

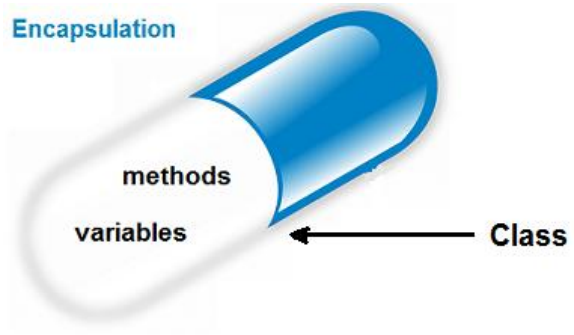
Anemic Model

- Bag of setters and getter
- No Behaviors included
- Considered as Anti-pattern
- Bulky services

```
public class Client
{
    public long Id { get; set; }

    public string FirstName { get; set; }

    public string LastName { get; set; }

    public string Status { get; set; }

    public ICollection<Invoice> Invoices { get; set; }
}
```

Rich Model

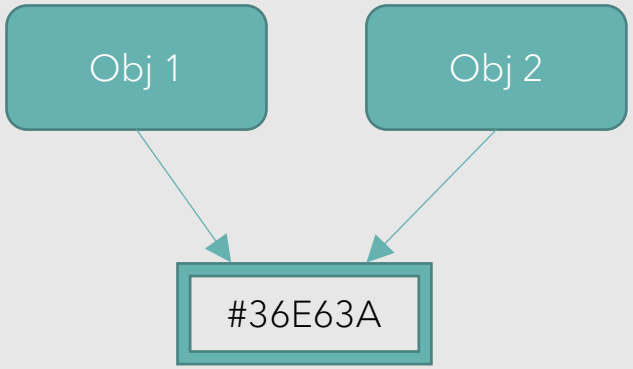
- Business logic inside domain models
- Apply OOP encapsulation principle
- Thin services
- Services only coordinate and delegate tasks to domain models



ENTITIES VS. VALUE OBJECTS

Types of Equality

Reference Equality	Identifier Equality	Structural Equality
Same reference	Same value of the ID field	All fields are matched



Obj 1
ID: 15

Obj 2
ID: 15

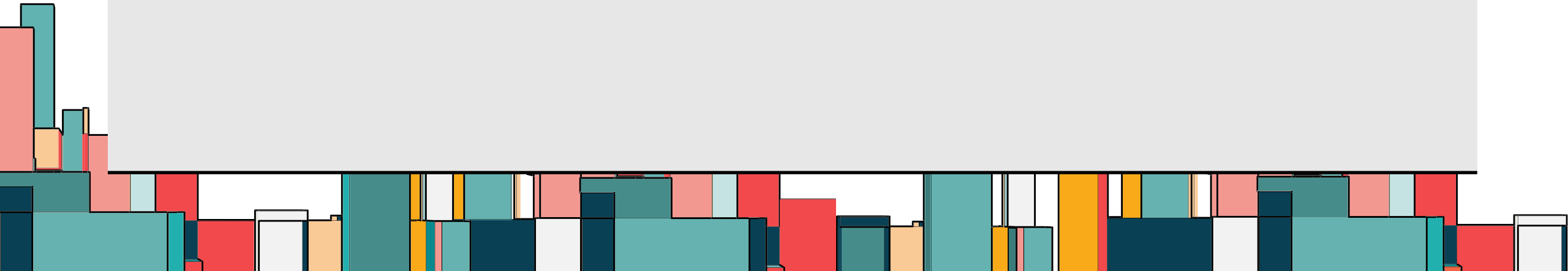
Obj 1
Year: 2019
Month: 08
Day: 14

Obj 2
Year: 2019
Month: 08
Day: 14

Entities Value Objects Entities Value Objects

ENTITIES VS. VALUE OBJECTS

Entities	Value Objects
• Identity based on ID field • Mutable • May have methods • Continuum • Represent system state	• Identity based on composition of the values • MUST be Immutable • May have methods • Have a zero lifespan • Have a meaning only in the context of an entity



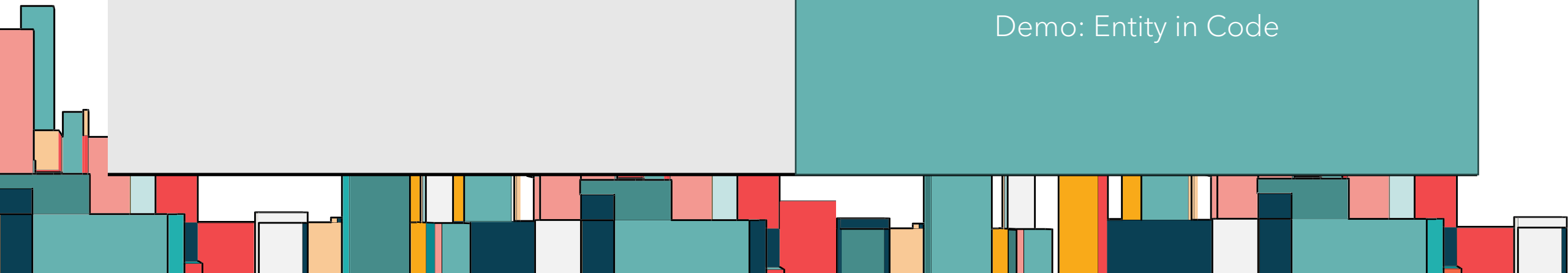
ENTITIES VS. VALUE OBJECTS

Entities

- Identity based on ID field
- Mutable
- May have methods
- Continuum
- Represent system state

Single Responsibility is a good principal to apply to entities

Demo: Entity in Code



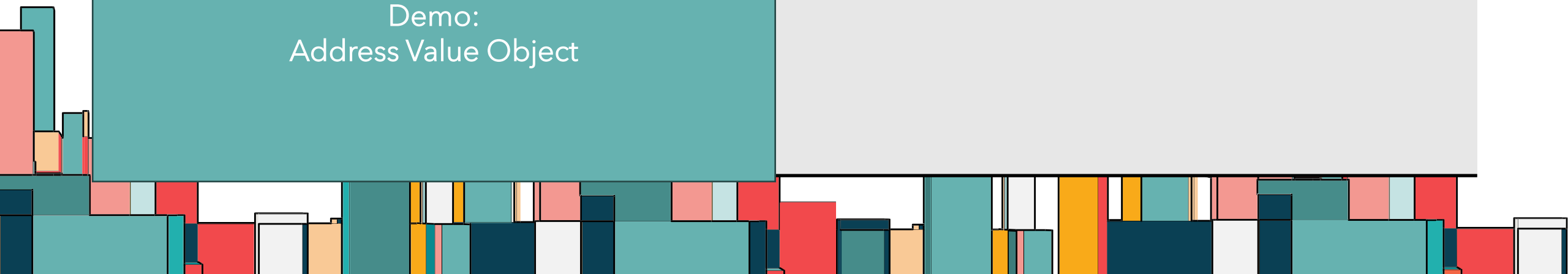
ENTITIES VS. VALUE OBJECTS

Examples: string & DateTime

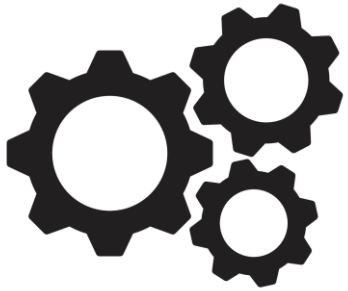
Demo:
Address Value Object

Value Objects

- Identity based on composition of the values
- MUST be Immutable
- May have methods
- Have a zero lifespan
- Have a meaning only in the context of an entity



DOMAIN SERVICES



- Stateless classes
- Encapsulates domain logic or operations that don't naturally fit within the boundaries of an Entity or a Value Object.
- Example: calculate the shipping cost for an order

Demo: Shipping Calculator